

PART 3 — COMPLETE REACT JS MASTER NOTES

React Basic → Advanced | JSX • Components • Props • State • Hooks • Routing • APIs • Architecture

Detailed standalone handbook: Basic → Intermediate → Advanced, with syntax, complete examples, expected output/behavior, debugging, projects, interview and viva preparation.

ROADMAP

- React mental model
- JSX
- Components
- Props
- State
- Events
- Forms
- Refs
- useEffect
- useContext
- useMemo/useCallback
- useRef
- Custom Hooks
- useDebugValue
- Routing
- API integration
- Performance
- Architecture
- Testing concepts
- Projects

1. React Foundation

React is a library for building component-based user interfaces. Components receive inputs (props), maintain state when needed and return UI descriptions.

Create a component

```
function Welcome() {  
  return <h1>Hello React</h1>;  
}  
  
export default Welcome;
```

Render root

```
import { createRoot } from "react-dom/client";  
import App from "./App";  
  
createRoot(document.getElementById("root"))  
  .render(<App />);
```

Project structure

```
src/  
  components/  
  pages/  
  hooks/  
  services/  
  App.jsx  
  main.jsx
```

Key point: A component should generally have one clear responsibility and a predictable input/output relationship.

2. JSX

JSX combines JavaScript expressions with HTML-like syntax. It is transformed into JavaScript before the browser executes it.

JSX basics

```
function App() {  
  const name = "Aryan";  
  
  return (  
    <main>  
      <h1>Hello {name}</h1>  
      <p>2 + 3 = {2 + 3}</p>  
    </main>  
  );  
}
```

className and style

```
<div className="card">  
  <h2 style={{ color: "navy" }}>Title</h2>  
</div>
```

Conditional rendering

```
{isLoggedIn ? (  
  <Dashboard />  
) : (  
  <Login />  
)}  
  
{items.length > 0 && <List items={items} />}
```

Lists and keys

```
items.map(item => (  
  <li key={item.id}>{item.name}</li>  
))
```

3. Components, Props and State

Components compose applications. Props flow from parent to child; state is local reactive data.

Props

```
function UserCard({ name, role }) {
  return (
    <article>
      <h2>{name}</h2>
      <p>{role}</p>
    </article>
  );
}

<UserCard name="Aryan" role="Developer" />
```

useState

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(c => c + 1)}>
      {count}
    </button>
  );
}
```

State object

```
const [user, setUser] = useState({
  name: "",
  age: 0
});

setUser(prev => ({
  ...prev,
  name: "Aryan"
}));
```

Key point: Never mutate React state directly. Create a new value and pass it to the setter.

4. Events and Forms

React event handlers receive synthetic event objects and forms are commonly controlled with state.

Events

```
function Button() {
  const handleClick = event => {
    console.log(event.currentTarget);
  };

  return <button onClick={handleClick}>Save</button>;
}
```

Controlled input

```
const [email, setEmail] = useState("");

<input
  type="email"
  value={email}
  onChange={e => setEmail(e.target.value)}
/>
```

Submit

```
function handleSubmit(e) {
  e.preventDefault();
  console.log(email);
}
```

```
}
```

Refs

```
const inputRef = useRef(null);

function focusInput() {
  inputRef.current?.focus();
}


```

5. Hooks

Hooks let function components use state, effects, context and other React features.

Rules of Hooks

- Call Hooks only at the top level of a React function component or custom Hook.
- Do not call Hooks inside loops, conditions or nested functions.
- Use dependency arrays correctly for effects and memoization.

useEffect

```
useEffect(() => {
  document.title = `Count: ${count}`;

  return () => {
    // cleanup
  };
}, [count]);
```

useContext

```
const ThemeContext = createContext("light");

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() {
  const theme = useContext(ThemeContext);
  return <div>{theme}</div>;
}
```

useMemo

```
const expensiveValue = useMemo(
  () => expensiveCalculation(items),
  [items]
);
```

useCallback

```
const handleSave = useCallback(
  () => save(id),
  [id]
);
```

useRef

```
const renderCount = useRef(0);
renderCount.current += 1;
```

6. Custom Hooks and Debugging

Custom Hooks package reusable stateful behavior.

Custom Hook

```
function useDocumentTitle(title) {
  useEffect(() => {
    document.title = title;
  }, [title]);
}

function Page() {
  useDocumentTitle("Dashboard");
  return <h1>Dashboard</h1>;
}
```

useDebugValue

```
function useOnlineStatus() {
  const [online, setOnline] = useState(navigator.onLine);

  useDebugValue(online ? "Online" : "Offline");

  return online;
}
```

Debugging

- React DevTools can inspect component trees and props/state.
- Check console warnings for keys, effects and controlled inputs.
- Trace network calls separately from rendering problems.
- Split large components when debugging becomes difficult.

7. Routing

Client-side routing maps URLs to React UI without full document reloads in typical SPA configurations.

Basic router

```
import {
  BrowserRouter,
  Routes,
  Route,
  Link
} from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

URL parameter

```
<Route
  path="/users/:id"
  element={<User />}
/>

function User() {
  const { id } = useParams();
  return <p>User {id}</p>;
}
```

8. Data Fetching and API Integration

Keep API access separate when applications grow. Handle loading, success and error states explicitly.

Fetch in an effect

```
const [users, setUsers] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);

useEffect(() => {
  let cancelled = false;

  async function load() {
    try {
      const res = await fetch("/api/users");
      if (!res.ok) throw new Error("Request failed");
      const data = await res.json();

      if (!cancelled) setUsers(data);
    } catch (err) {
      if (!cancelled) setError(err.message);
    } finally {
      if (!cancelled) setLoading(false);
    }
  }

  load();
  return () => { cancelled = true; };
}, []);
```

9. Performance and Architecture

Optimize based on evidence. Component boundaries, stable keys, correct effects and avoiding unnecessary work usually matter more than premature memoization.

Memoized component

```
const UserRow = memo(function UserRow({ user }) {
  return <li>{user.name}</li>;
});
```

Lazy loading

```
const Settings = lazy(
  () => import("../pages/Settings")
);

<Suspense fallback={<p>Loading...</p>}>
  <Settings />
</Suspense>
```

Architecture

src/	
components/	reusable UI
pages/	route-level screens
hooks/	custom hooks
services/	API clients
context/	shared providers
utils/	pure helpers
assets/	static assets

10. Advanced React Concepts

Understand rendering, reconciliation, state ownership, context tradeoffs and error boundaries.

State lifting

Move state to the nearest common ancestor when multiple children need to coordinate. Pass state and event handlers downward as props.

Composition

```
function Card({ children }) {  
  return <section className="card">{children}</section>;  
}  
  
<Card>  
  <h2>Profile</h2>  
  <p>Content</p>  
</Card>
```

Error boundary concept

Error boundaries catch rendering errors in descendant trees using the supported class-based boundary pattern or framework-level mechanisms.

Key point: Avoid putting every value into Context. Context is useful for genuinely shared values such as theme, locale or authentication state, but frequent updates can create broad re-rendering.

PRACTICAL PROJECTS

Project 1 — Student Manager

CRUD-style component architecture.

```
function StudentList({ students, onDelete }) {  
  return (  
    <ul>  
      {students.map(student => (  
        <li key={student.id}>  
          {student.name}  
          <button onClick={() => onDelete(student.id)}>  
            Delete  
          </button>  
        </li>  
      )  
    )  
  )  
};
```

Expected result/preview: run the example in the stated environment. The visible result should match the described component or behavior. Use DevTools/console/network tools to verify it.

Project 2 — Search UI

Controlled input + derived filtering.

```
const [query, setQuery] = useState("");  
  
const filtered = students.filter(student =>  
  student.name.toLowerCase().includes(query.toLowerCase())  
);
```

Expected result/preview: run the example in the stated environment. The visible result should match the described component or behavior. Use DevTools/console/network tools to verify it.

Project 3 — API Dashboard

Loading/error/success states with reusable components.

```
function Status({ loading, error, children }) {  
  if (loading) return <p>Loading...</p>;  
  if (error) return <p role="alert">{error}</p>;  
  return children;  
}
```

Expected result/preview: run the example in the stated environment. The visible result should match the described component or behavior. Use DevTools/console/network tools to verify it.

INTERVIEW + VIVA REVISION

- What is React?
- Why JSX?
- Component vs element?
- Props vs state?
- Why keys?
- Controlled vs uncontrolled inputs?
- Rules of Hooks?
- useState behavior?
- useEffect dependencies?
- Cleanup function?
- useContext?
- useMemo vs useCallback?
- useRef?
- Custom Hook?
- Why avoid state mutation?
- What is lifting state?
- What is composition?
- Why use routing?
- What is lazy loading?
- What causes re-render?
- How do you optimize React?
- What is memo?
- How do you debug React?
- How do you structure a React project?
- How do you handle API loading/error states?

FINAL CHEAT SHEET

Revise the syntax patterns, lifecycle/flow, common errors, and project architecture. The goal is not memorizing every property or method, but knowing the model well enough to build and debug applications.